Unit-5

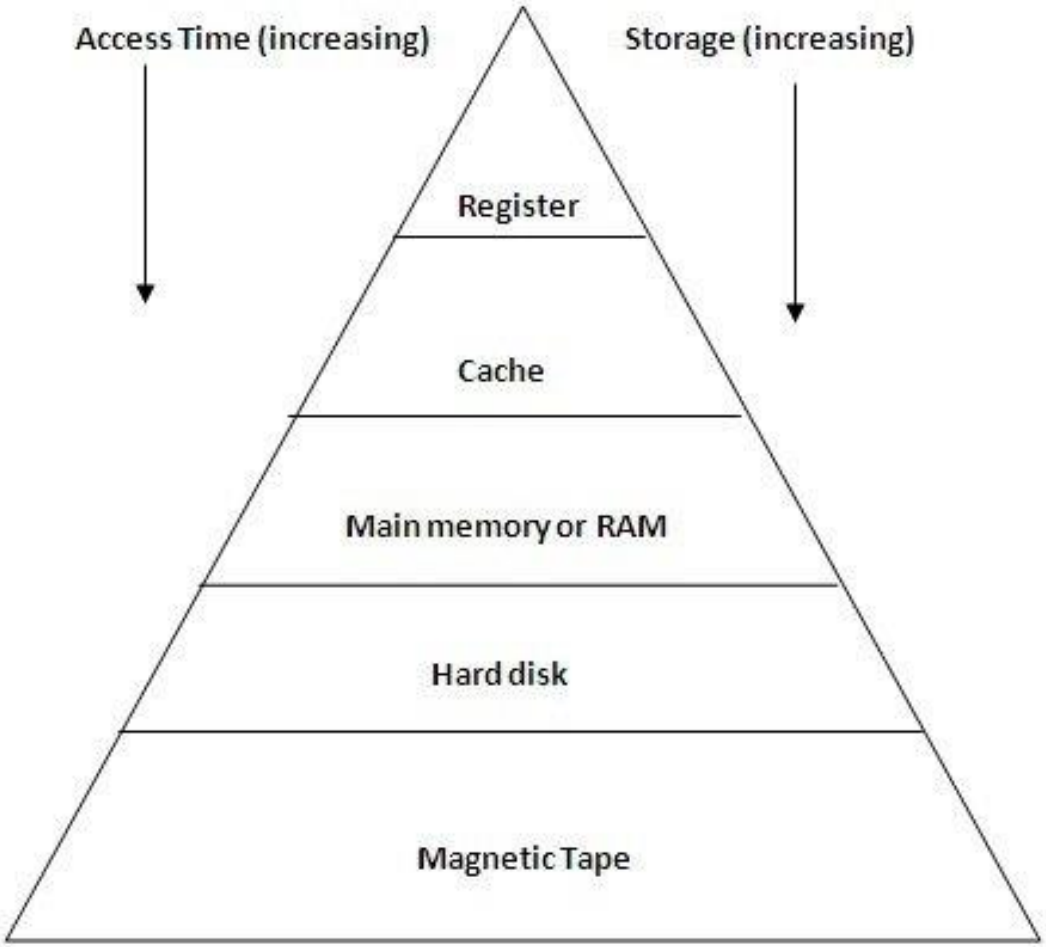
Memory Management – Part A

Introduction

- Memory is used to store data and instructions either permanently or temporarily during processing.
- It consists of a large array of words or bytes, each with its own address.
- Computer memory can be defined as a collection of some data represented in the binary format. On the basis of various functions, memory can be classified into various categories.

Memory Hierarchy

- The memory in a computer can be divided into five hierarchies based on the speed as well as use.



Memory Hierarchy (contd..)

- Register is a small memory location present inside processor. It is typically 64 or 128 bits. It is used to store data and instructions during processing.
- Cache memory is a high speed memory present in between RAM and CPU. It stores those data and instructions which are frequently required by CPU for processing.
- Main memory is used for storing data throughout the operations of the computer. It communicates with the CPU directly.
- Magnetic disk is a secondary memory. It stores data in the circular plates made of metal or plastic.
- Magnetic tape contains thin plastic ribbon to store data. It is mainly used to backup huge data.

Logical versus Physical Address space

- The memory address is used to identify or locate a particular memory location in the computer.
- Memory addresses are of two types:

1) Logical Address
2) Physical Address

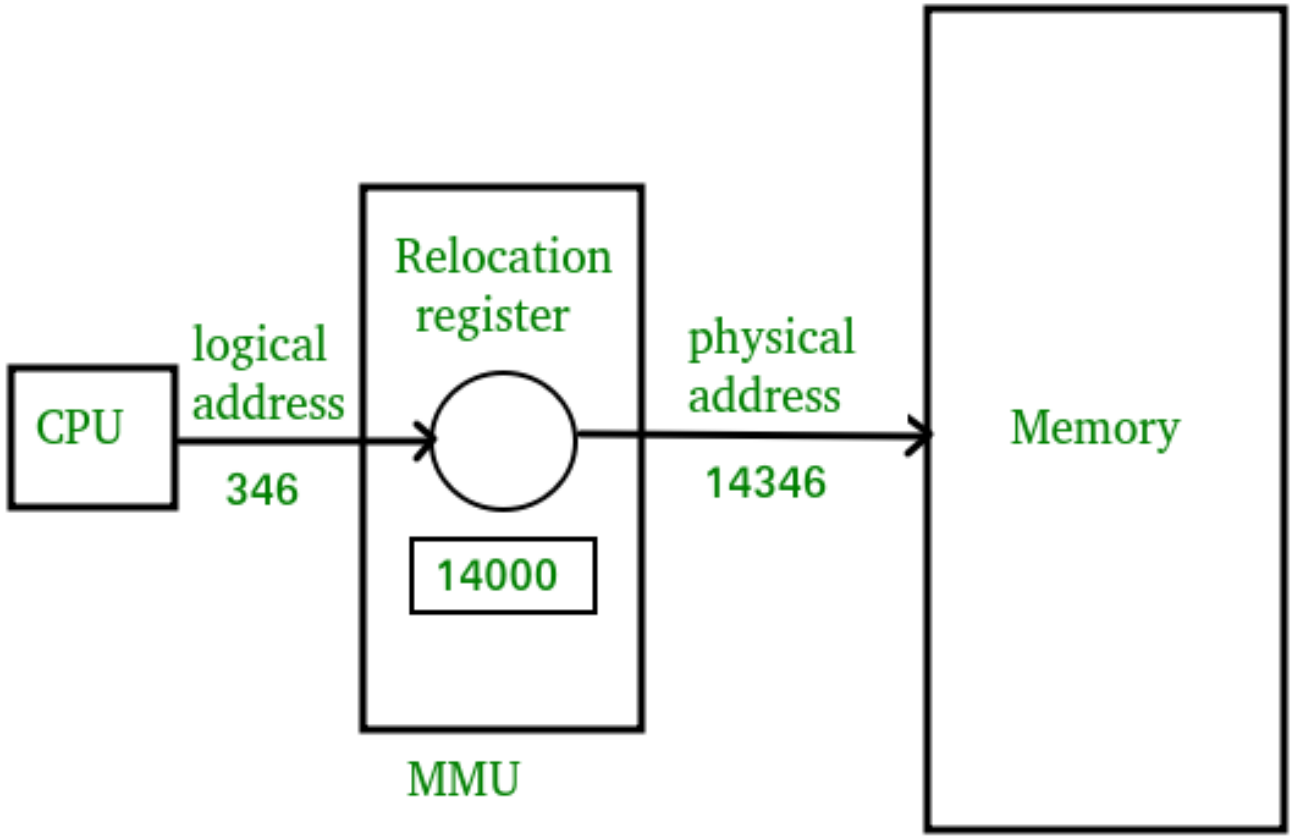
Logical Address

- It is the address generated by CPU while a program is running.
- It is virtual as it does not exist physically.
- It is used as a reference to access the physical memory location.
- The logical address is mapped to its corresponding physical address by a hardware device called Memory-Management Unit (MMU).
- The address-binding methods used by MMU generates identical logical and physical address during compile and load time but generates different logical and physical address during run-time.

Physical Address

- It identifies a physical location in a memory.
- MMU (Memory-Management Unit) computes the physical address for the corresponding logical address.
- The user never deals with the physical address.
- Instead, the physical address is accessed by its corresponding logical address by the user.
- The user program generates the logical address, but the program needs physical memory for its execution. Hence, the logical address must be mapped to the physical address before they are used.

Logical to physical address mapping



Logical versus Physical Address

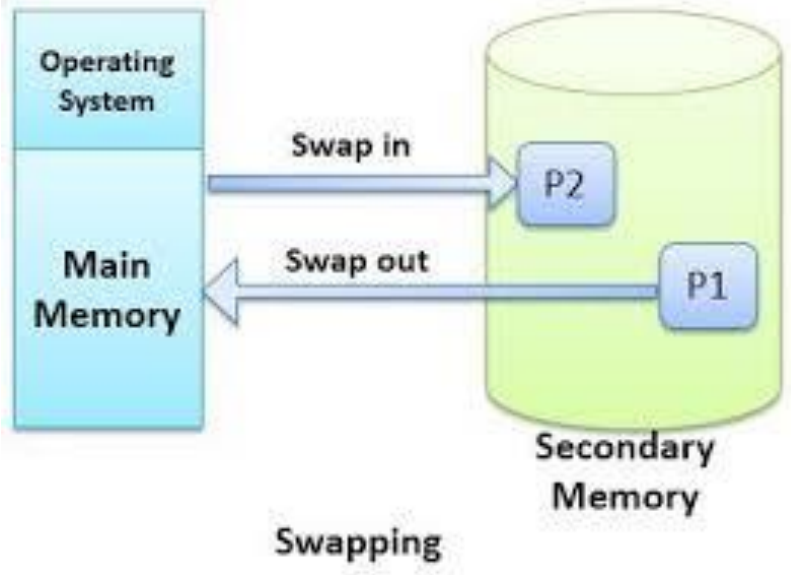
Logical Address	Physical Address
It is generated by CPU in perspective of a program	It is a location that exists in the memory unit.
The set of all logical addresses generated by CPU for a program is called logical address space.	The set of all physical address mapped to corresponding logical address is referred as physical address space.
It is called as virtual address as the address doesn’t exists physically in the memory unit.	The physical address is a location in the memory unit that can be accessed physically.
The logical address is generated by the CPU while program is running.	The physical address is computed by the MMU (Memory Management Unit)

Memory Management strategies

- Memory management with swapping
- Memory management without swapping
- Memory management with bitmaps
- Memory management with linked lists

Memory Management with Swapping

- It is a mechanism in which a process can be swapped out temporarily of main memory to secondary memory and make the memory available to other processes.
- The system swaps back the process from the secondary memory to main memory later.



Swapping

Memory Management with Swapping (contd..)

- It is used to improve the performance of the system. The process which is suspended or waiting for resource/input is swapped out and the process which is ready for execution is swapped in.

- It is done by the program called as sweeper. The sweeper is used for:
  - Selecting which process to be out
  - Selecting which process to be in
  - Providing the memory space to the processes those are newly entered

Memory management without swapping

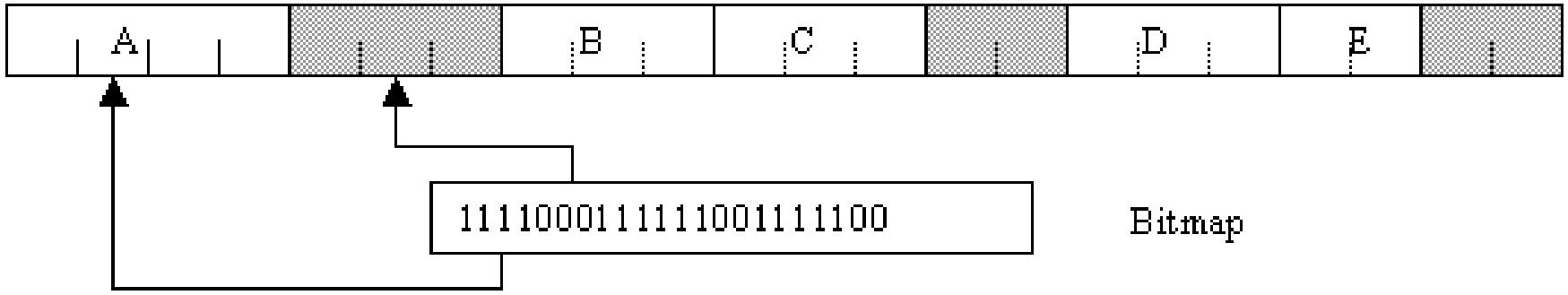
- In this scheme of memory management, the entire process remains in memory from start to finish and doesn’t move.

- The sum of the memory requirements of all jobs in the system cannot exceed the size of physical memory.

Memory management with Bitmaps

- With bitmap, memory is divided into allocation units.

- Corresponding to each allocation unit there is a bit in a bitmap.
  - Bit is 0 if the unit is free.
  - Bit is 1 if unit is occupied.



Memory management with Bitmaps (contd..)

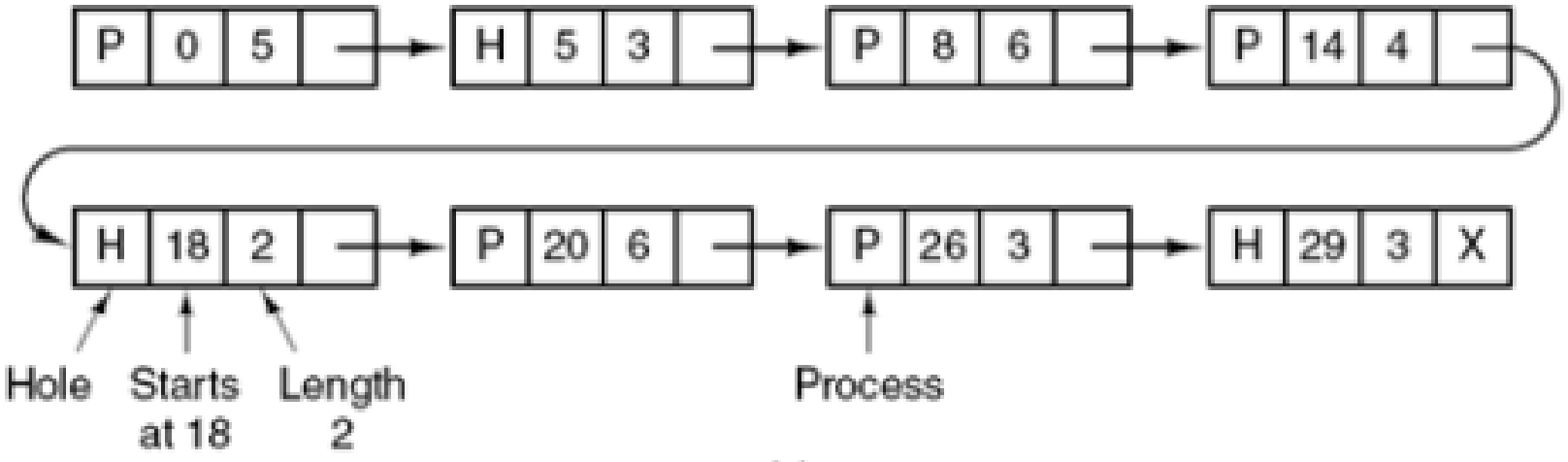
- The size of allocation unit is an important design issue:
  - the smaller the size, the larger the bitmap.
  - If we use larger allocation unit, we could waste memory as we may not use all the space allocated in each allocation unit.

- The other problem with this approach is when we need to allocate memory to a process. Assume the allocation size is 4 bytes. If a process requests 256 bytes of memory, we must search the bit map for 64 consecutive 0's.

- This is a slow operation and for this reason bit maps are not often used.

Memory management with Linked Lists

- Linked list can be used to keep track of memory of allocated and free memory segments, where a segment is either a process or a hole between two processes.



Memory management with Linked Lists (contd..)

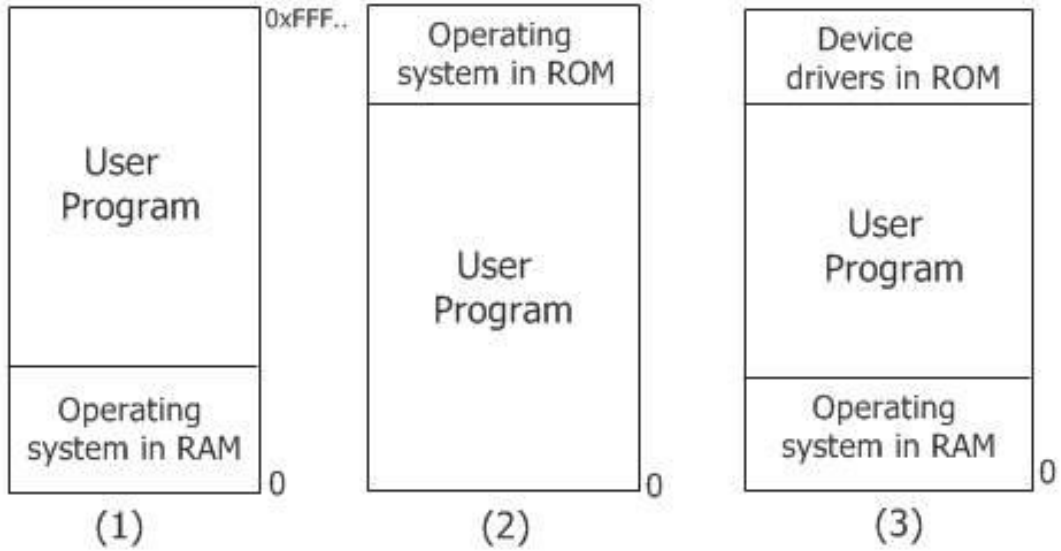
- Each entry in the list specifies:
  - a hole (H) or process (P),
  - the address at which it starts,
  - the length and
  - a pointer to the next entry.

- It is easier to update the list when a new process is created or terminated.

Mono-programming/Uniprogramming

- It contains only one program at any point of time.
- Most of the memory capacity is dedicated to a single program. Only a small part is needed to hold the operating system.
- When the program finishes execution, the program area is occupied by another program.
- It is not efficient in terms of memory utilization.

Mono-programming models



- The first model is rarely used now-a-day, but was formally used on mainframes and minicomputers.
- The second model is used on some palmtop computers and embedded systems.
- Third model was used by early PC, for example MS-DOS.

Mono-programming

- The characteristics of Mono-programming are:
  - It allows only one program to be present in memory at a time.
  - The resources are provided to the single program that is present in the memory at that time.
  - Since only one program is loaded, the size is small as well.

- Example of Mono-programming
  - Batch processing in old computers and mobiles
  - The old operating system of computers
  - Old mobile operating systems.

Multiprogramming

- Multiple programs reside in main memory at a time.
- At present, almost all computer system allow multiprogramming.
- Multiprogramming increases the CPU utilization. When multiple processes are running at the same time, means that when process is blocked waiting for input/output to finish, another one can use the CPU.
- To achieve multiprogramming, the simplest way is just to divide memory into n partitions.
- It can be implemented in following two ways:
  - Dedicated partitions for each process (Absolute translation)
  - Maintaining a single queue (Re-locatable translation)

Multiprogramming (contd..)

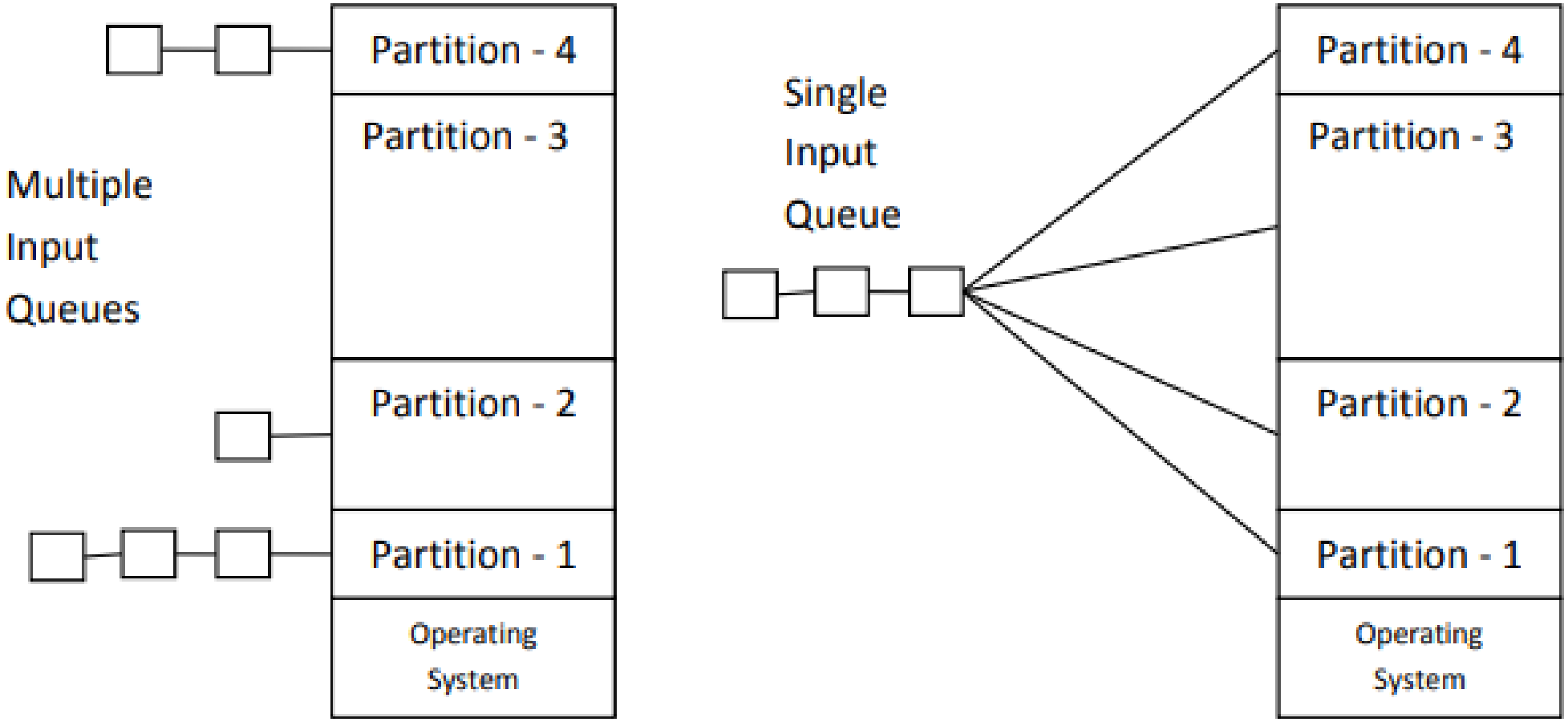
- Example: Modern OS like Windows XP, 7, 8, 10.
- Characteristics:
  - Multiple programs can be present in the memory at a given time.
  - The resources are dynamically allocated.
  - The size of the memory is comparatively larger.

Multiprogramming (contd..)

- Advantages:
  - CPU utilization is better.
  - System looks faster as all the tasks runs in parallel.
  - Supports multiple users
  - Response time is shorter

- Disadvantages:
  - It is difficult to program a system because of complicated schedule handling.
  - Tracking all tasks/process is difficult.
  - Due to high load of tasks, long time jobs have to wait long.

Dedicated partitions for each process vs Maintaining a single queue



Dedicated partitions for each process

- Separate input queue are maintained for each partition.
- The main problem of this implementation is: if a process is ready to run and its partition is occupied then that process has to wait even if other partitions are available.
- Wastage of storage.

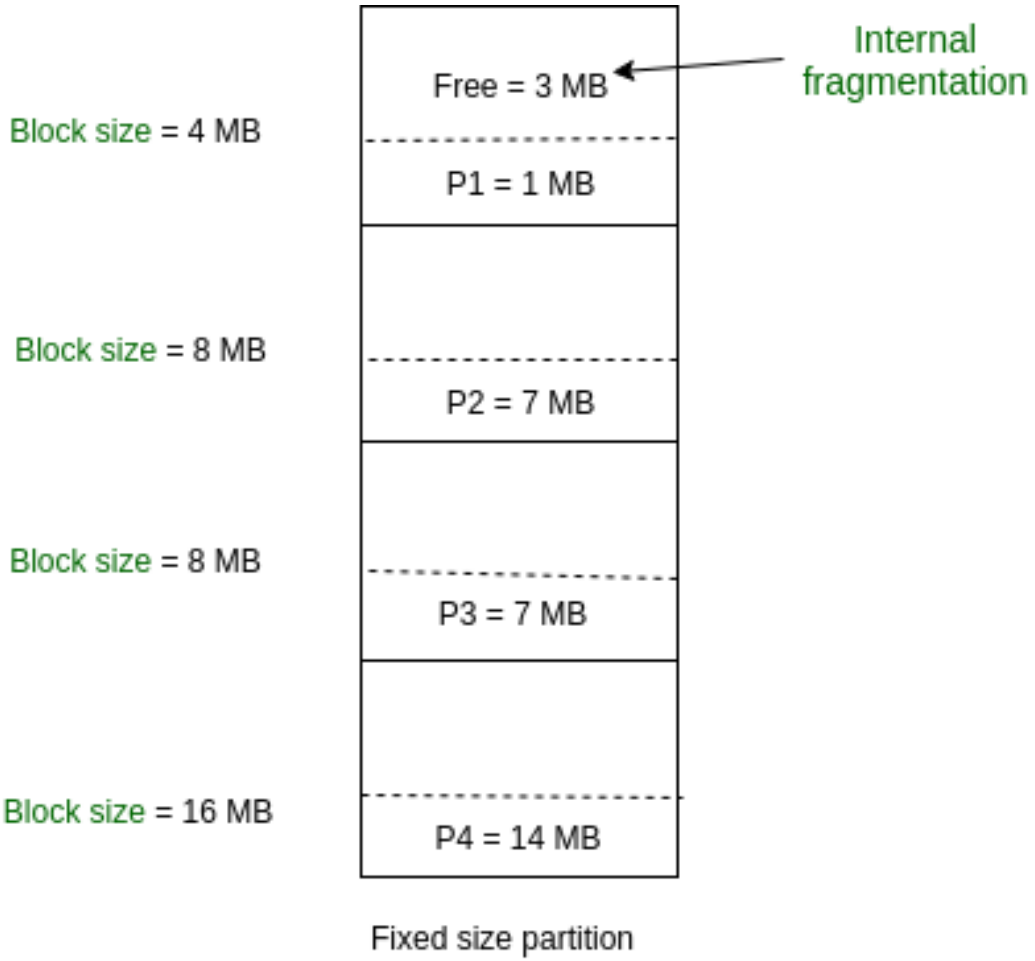
Maintaining a single queue

- A single queue is maintained for all processes.
- When a partition becomes free, the process closest to the front of queue that fits in it could be loaded into the empty partition and run.
- To minimize the waste of large partition to small process, another strategy is to search the whole input queue whenever a partition becomes free and pick the largest process that fits.
- Problems:
  - Eliminates the problem of dedicated partition for each process but it is complex to implement.
  - Wastage of storage when many processes are small.

Multiprogramming with Fixed and Variable Partitions

- There are two Memory Management Techniques: Contiguous and Non-Contiguous.
- **Contiguous memory allocation** allocates consecutive blocks of memory to a file/process.
- **Non-Contiguous memory allocation** allocates separate blocks of memory to a file/process.
- Contiguous technique can be divided into:
  - Fixed (or static) partitioning
  - Variable (or dynamic) partitioning

Fixed (or static) Partitioning



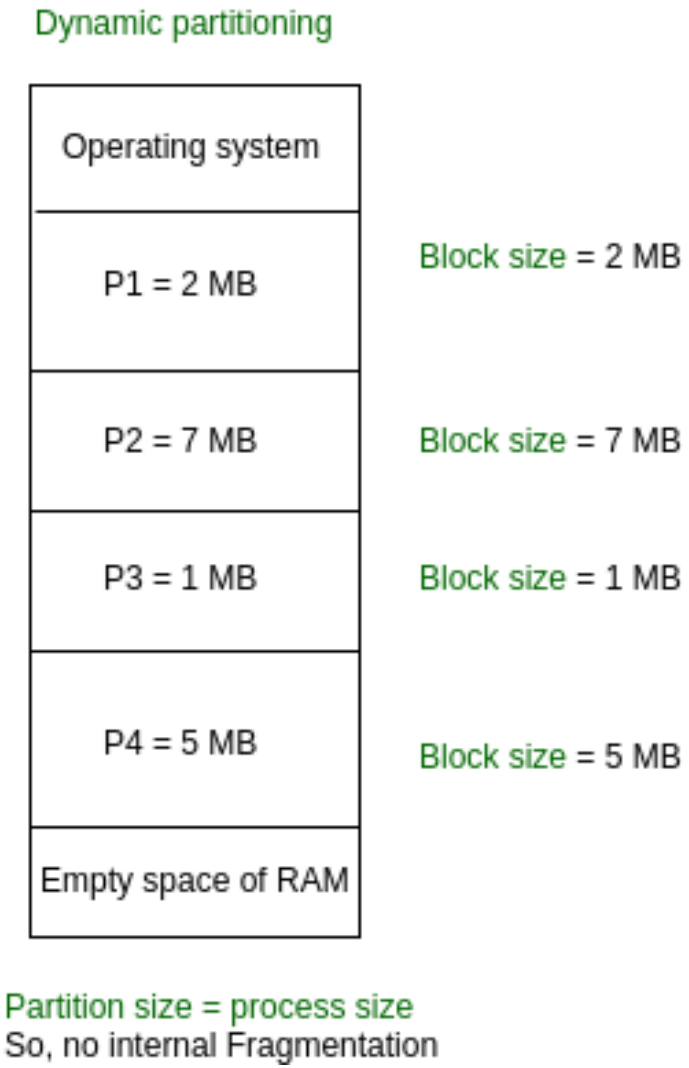
Fixed (or static) Partitioning (contd..)

- It is the oldest and the simplest technique used to place more than one processes in the main memory.
- In this technique, the number of partitions in RAM is fixed but size of each partition may or may not be same.
- The partitions are created before execution or during system configure.
- The total of Internal fragmentation in every block = 3+1+1+2 = 7MB
- Advantages:
  - Easier to implement
  - Less OS overhead to create and manage partition

Fixed (or static) Partitioning (contd..)

- Disadvantages:
  - Inefficient in terms of memory utilization due to internal fragmentation.
  - External fragmentation: the total unused space of various partitions cannot be used to load the processes.
  - Limit process size: Process of size greater than size of partition in main memory cannot be accommodated.
  - Limitation on degree of Multiprogramming: Number of processes greater than number of partitions in RAM are invalid.

Variable (or dynamic) partitioning



Variable (or dynamic) partitioning (contd..)

- In contrast with fixed partitioning, partitions are not made before the execution or during system configure.

- Various features associated with variable Partitioning:
  - Initially RAM is empty and partitions are made during the run-time according to process's need instead of partitioning during system configure.
  - The size of partition will be equal to incoming process.
  - The partition size varies according to the need of the process so that the internal fragmentation can be avoided to ensure efficient utilization of RAM.
  - Number of partitions in RAM is not fixed and depends on the number of incoming process and Main Memory's size.

Variable (or dynamic) partitioning (contd..)

- Advantages:
  - No internal fragmentation
  - No restriction on degree of multiprogramming
  - No limitation on the size of the process

- Disadvantages:
  - Complex to implement
  - External fragmentation can still exist.

Fixed partitioning	Variable partitioning
In multi-programming with fixed partitioning the main memory is divided into fixed sized partitions.	In multi-programming with variable partitioning the main memory is not divided into fixed sized partitions.
Only one process can be placed in a partition.	In variable partitioning, the process is allocated a chunk of free memory.
It does not utilize the main memory effectively.	It utilizes the main memory effectively.
There is presence of internal fragmentation and external fragmentation.	There is external fragmentation.
Degree of multi-programming is less.	Degree of multi-programming is higher.
It is more easier to implement.	It is less easier to implement.
There is limitation on size of process.	There is no limitation on size of process.

Fragmentation

- Fragmentation is an unwanted problem where the memory blocks cannot be allocated to the processes due to their small size and the blocks remain unused.
- It can also be understood as when the processes are loaded and removed from the memory they create free space or hole in the memory and these small blocks cannot be allocated to new upcoming processes and results in inefficient use of memory.
- Types:
  - Internal Fragmentation
  - External Fragmentation

Internal Fragmentation

- In this fragmentation, the process is allocated a memory block of size more than the size of that process. Due to this some part of the memory is left unused and this cause internal fragmentation.

- The difference between memory allocated and required space or memory is called Internal fragmentation.

- It occurs when memory is divided into fixed sized partitions. It can be solved by using variable or dynamic partitions.

External Fragmentation

- In this fragmentation, although we have total space available that is needed by a process still we are not able to put that process in the memory because that space is not contiguous.
- This problem is occurring because we are allocating memory continuously to the processes. So, if we remove this condition external fragmentation can be reduced. This is what done in paging & segmentation.
- Another way to remove external fragmentation is compaction. When dynamic partitioning is used for memory allocation then external fragmentation can be reduced by merging all the free memory together in one large block. This technique is also called defragmentation.

Memory allocation strategies

- Memory allocation is the process of assigning blocks of memory on request.
- When the processes and holes are kept on a list sorted by address, several algorithms can be used to allocate memory for a newly created process
- Different algorithms used to allocate memory for a process (partition selection) are:
  - First Fit
  - Next Fit
  - Best Fit
  - Worst Fit
  - Quick Fit

First Fit

- Allocate the first hole that is big enough.
- The process manager scans along the list of segments until it finds a hole that is big enough. It stop the searching as soon as it find a free hole that is large enough.
- The hole is then broken up into two pieces, one for the process and one for unused memory.

Advantage: It is a fast algorithm because it search as little as possible.

Problem: not good in terms of storage utilization.

First Fit (Contd..)

Example: Given five memory partitions of 100Kb, 500Kb, 200Kb, 300Kb, 600Kb (in order), how would the first-fit algorithm place processes of 212 Kb, 417 Kb, 112 Kb, and 426 Kb (in order)?

Soln:

1. 212K is put in 500K partition
2. 417K is put in 600K partition
3. 112K is put in 288K partition (new partition 288K = 500K - 212K)
4. 426K must wait

Next Fit

- Next fit is a modified version of 'first fit'.

- It begins as the first fit to find a free partition but when called next time it starts searching from where it left off, not from the beginning.

Next Fit (contd..)

Example: Given five memory partitions of 100Kb, 500Kb, 200Kb, 300Kb, 600Kb (in order), how would the next-fit algorithm place processes of 212 Kb, 417 Kb, 112 Kb, and 426 Kb (in order)?

Best Fit

- The best fit deals with allocating the smallest free partition which meets the requirement of the requesting process.
- This algorithm first searches the entire list of free partitions and considers the smallest hole that is adequate. It then tries to find a hole which is close to actual process size needed.
- Rather than breaking up a big hole that might be needed later, best fit tries to find a hole that is close to the actual size needed.
- Advantage: Memory utilization is much better than first fit as it searches the smallest free partition first available.
- Disadvantage: It is slower and may even tend to fill up memory with tiny useless holes.

Best Fit (contd..)

Example: Given five memory partitions of 100Kb, 500Kb, 200Kb, 300Kb, 600Kb (in order), how would the best-fit algorithm place processes of 212 Kb, 417 Kb, 112 Kb, and 426 Kb (in order)?

Soln:

1. 212K is put in 300K partition
2. 417K is put in 500K partition
3. 112K is put in 200K partition
4. 426K is put in 600K partition

Worst Fit

- Worst Fit allocates a process to the partition which is largest sufficient among the available partitions in the main memory.
- It uses the concept of always take the largest available hole, so that the hole broken off will be big enough to be useful.

- Advantage: Reduces the rate of production of small gaps.
- Disadvantage: If a process requiring larger memory arrives at a later stage then it cannot be accommodated as the largest hole is already split and occupied.

Worst Fit (contd..)

Example: Given five memory partitions of 100Kb, 500Kb, 200Kb, 300Kb, 600Kb (in order), how would the worst-fit algorithm place processes of 212 Kb, 417 Kb, 112 Kb, and 426 Kb (in order)?

Soln:

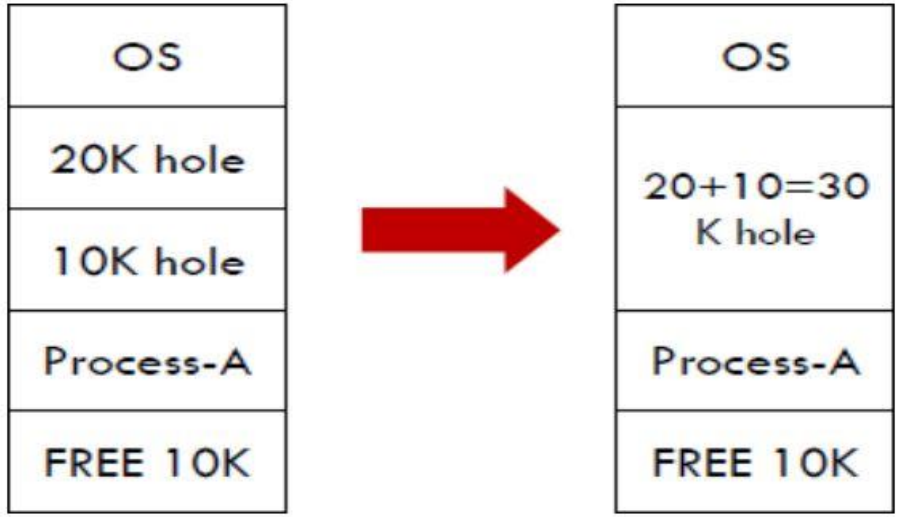
1. 212K is put in 600K partition
2. 417K is put in 500K partition
3. 112K is put in 388K partition
4. 426K must wait

Quick Fit

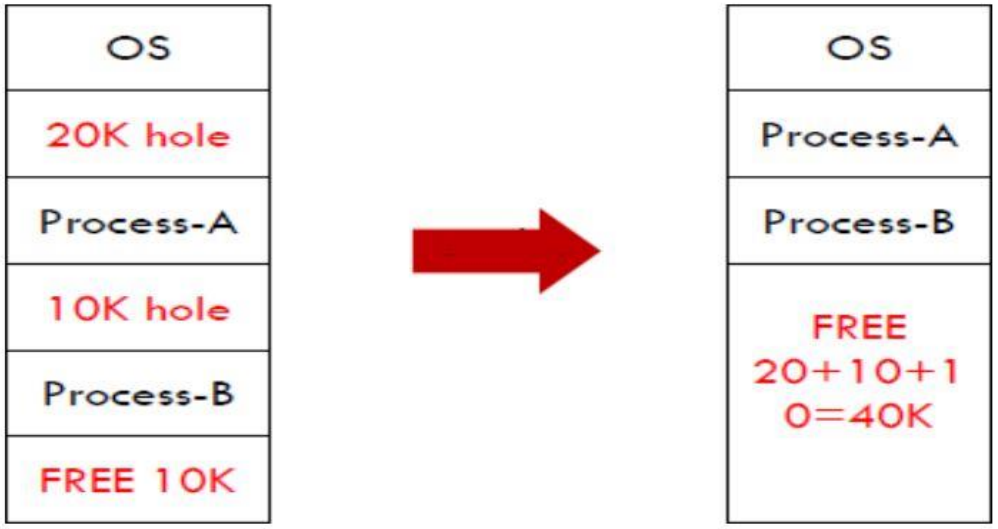
- The quick fit algorithm suggests maintaining the different lists of frequently used sizes (more common sizes).
- Although, it is not practically suggestible because the procedure takes so much time to create the different lists and then expending the holes to load a process.
- With quick fit, finding the hole of the required size is extremely fast.

Coalescing and Compaction

Coalescing



Compaction



Coalescing and Compaction (contd..)

- The process of merging adjacent holes to form a single larger hole is called **coalescing**. By **coalescing** holes, the larger possible contiguous blocks of storage can be obtained.

- Memory compaction is the process of combining all the holes into one big one by moving all the processes downward as far as possible.

- The technique of storage **compaction** involves moving all occupied areas of storage to one end.

Problem with compaction

- The efficiency of the system is decreased in the case of compaction due to the fact that all the free spaces will be transferred from several places to a single place.

- Huge amount of time is invested for this procedure and the CPU will remain idle for all this time. Despite of the fact that the compaction avoids external fragmentation, it makes system inefficient.

Relocation and Protection

- These are the problems that needs to be addressed when using multiprogramming.

Relocation :

- When a program is run it does not know in advance what location it will be loaded at.
- Programmers typically do not know in advance which other programs will be resident in main memory at the time of execution of their program.
- Active processes need to be able to be swapped in and out of main memory in order to maximize processor utilization

Protection :

- Once we have two or more programs in memory at the same time, there is a danger that one program can write to the address space of another program.
- This is obviously dangerous and should be avoided.

Relocation and Protection (contd..)

- A solution, which solves both the relocation and protection problem is to use two registers called the ***base*** and ***limit*** registers.
- The base register stores the start address of the partition and the limit register holds the length of the partition.
- Any address that is generated by the program has the base register added to it. In addition, all addresses are checked to ensure they are within the range of the partition.
- An additional benefit of this scheme is that if a program is moved within memory, only its base register needs to be amended. This is obviously a lot quicker than having to modify every address reference within the program.

Relocation and Protection (contd..)